NPU模型优化Agent Skill：端到端优化闭环实践

分享嘉宾：王子恒

2026.4.18　中国·北京

01 体系概述与效果样例

02 主流程编排与支撑约束机制

03 使用方式

04 未来规划

content

目录

体系概述与效果样例

01

BACKGROUND

通用 Agent 进行NPU 模型优化的典型问题

通用 Agent 难以稳定完成长链路、强依赖、重验证的 NPU 推理优化任务

仓库经验难复用，阶段选型不准确

真实优化依赖的不只是接口文档，还依赖仓库里的案例实现、推荐路径和踩坑经验，通用 Agent 缺少这些支撑时，常常不知道当前模型最该先做哪一步。

领域知识不完整，难以把握优化边界

通用 Agent 往往知道有哪些 CANN/NPU 优化能力，但不清楚它们各自的适用前提、组合关系和工程约束，因此容易做出表面合理但实际不适配的优化决策。

缺少分层上下文与结构化流程支撑

NPU 优化是一个多阶段复杂任务，如果没有系统化编排流程、明确前后依赖和验证节点，Agent 很容易在复杂信息中失焦，把任务做成反复试错。

验证门禁与回退机制不强，闭环难以稳定落地

局部阶段看起来有效，并不代表整条优化链路一定能走通，缺少逐阶段验证、异常回退和系统排查机制时，最终结果往往难以稳定收敛。

需要把优化经验沉淀为可执行、可验证、可复用的 Skill 工作流

Skill 工作流的核心价值：效率提升、门槛降低

SKILL SYSTEM OVERVIEW

Skill 体系全景

核心思路：通过一个主流程把 NPU 推理优化，拆分编排成明确的步骤，按阶段推进、验证。

开源仓库

- cann-recipes-infer 模型推理优化开源仓
  https://gitcode.com/cann/cann-recipes-infer
- CANNBot Agent Skills 工具开源仓
  https://gitcode.com/cann/skills

4 个核心优化阶段

并行化改造	KVCache+FA
融合算子	图模式

进阶优化 Skill： **model-infer-multistream** / **model-infer-**prefetch / **model-infer-**superkernel

当前模型优化 Skill 体系组成

1 编排层

model-infer-optimize
总入口，编排所有优化流程

2 核心层

model-infer-migrator
框架适配+部署基线

model-infer-parallel-analysis
并行策略分析

model-infer-parallel-impl
并行切分改造

model-infer-kvcache
KVCache+FA替换

model-infer-fusion
融合算子替换

model-infer-graph-mode
图模式适配

3 调试层

model-infer-precision-debug
精度问题调试诊断

model-infer-runtime-debug
NPU运行时报错、卡死诊断

VALIDATION RESULTS

验证模型覆盖与效果样例

Skill 体系已在多种模型与场景中完成端到端优化测试，性能与成功率显著优于无 Skill 对比

模型端到端优化测试

吞吐提升基于未优化的基线比较

Qwen3-0.6B 单卡，Dense GQA	**8.9x** Decode吞吐提升	**Qwen3.5-0.8B** 单卡，混合架构	**5.1x** Decode吞吐提升
Deepseek-OCR 单卡，多模态 MoE	**3.2x** Decode吞吐提升	**Nemotron** 单卡，Dense	**3.3x** Decode吞吐提升
DINOv3 单卡，ViT视觉	**3.4x** Decode吞吐提升	**Gemma-4** 8卡，MoE，共享KV	**5.4x** Decode吞吐提升
LongCat-Flash-Lite 8卡，MoE+MLA	**14.4x** Decode吞吐提升	**LongCat-Next** 8卡，MoE+MLA	**12.5x** Decode吞吐提升

单卡简单模型约2-3小时完成优化适配
多卡复杂模型约6-8小时完成优化适配

Qwen3.5 - 0.8B 优化结果对比样例

维度	使用 Skill	未使用 Skill
Decode 加速	**5.1x** (73.91→14.43ms)	**1.1x** (74.05→68.82ms)
Prefill 加速	**1.1x** (319.87→290.04ms)	**1.0x** (323.25→321.87ms)
E2E 吞吐提升	**2.8x** (11.2→31.59 tok/s)	**1.06x** (11.16→11.81 tok/s)
融合算子覆盖	全覆盖**7类**融合算子	仅覆盖**3类**融合算子
图模式	**成功**（解决 6 个图中断）	**放弃**（误判不可行）

对比说明：无 Skill 指相同环境下仅使用通用 Agent 与相同流程提示词

主流程编排与支撑约束机制

02

MAIN ORCHESTRATION

主流程编排

主流程编排将端到端优化组织为 6 个主要阶段

0 模型分析 + 基线建立
分析模型架构，完成部署并建立基线

1 并行化改造
分析各阶段并行化方案与并行切分改造

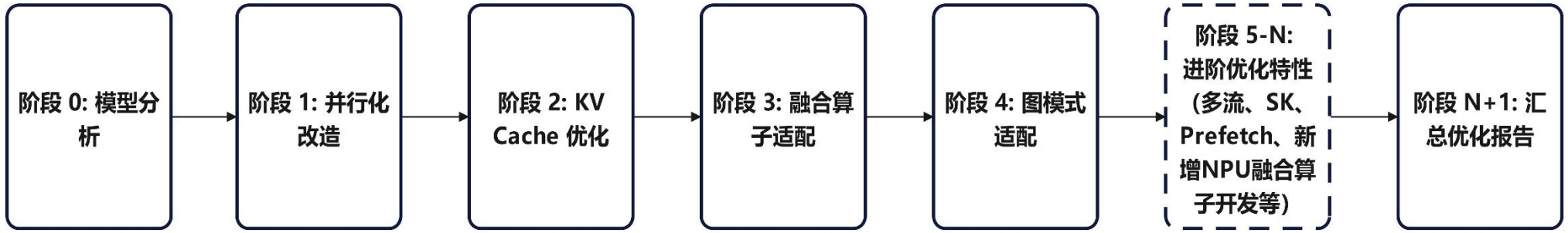
2 KVCache + FA
kvcache 静态化改造与适配FA算子

3 融合算子
融合算子分析匹配与实施替换

4 图模式
图模式分析、断点消除与实施适配

5 优化总结，生成报告
汇总优化成果，输出完整报告

整体优化skill流程



典型阶段流程模式

每个阶段严格遵循门禁流程

分析 → 方案确认 → 实施 → 自验证 → reviewer 验证 → 阶段总结

STAGE DETAILS

阶段详解：分工与角色派发

Analyzer
分析者

- ✓ 负责架构分析与方案设计
- ✓ 输出方案到 progress.md
- ✕ **禁止修改代码**

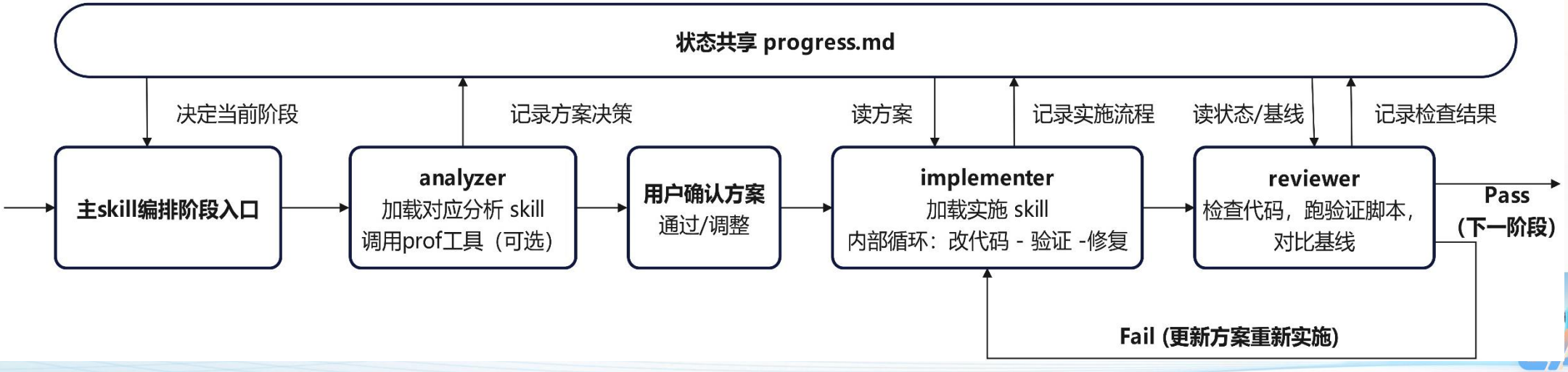
Implementer
实施者

- ✓ 负责代码改造与修复
- ✓ 读写全部文件
- ! **必须完成自验证**

Reviewer
验证者

- ✓ 负责代码审查与精度/性能验证
- ✕ **禁改模型代码**
- ✓ 输出验证报告

阶段1-N内部流程



ARCHITECTURE SUPPORT

架构支撑

分层上下文

1 AGENT.md

每次加载，≤100行，项目概述与通用约束

2 Skills

按需加载，≤500行，领域流程与规则

3 References / Scripts

流程指定读取，不限体量，配置与示例

状态管理

跨会话断点接力

progress.md

常驻区+工作区

分阶段归档

阶段推进时归档清空

HARNESS SYSTEM

Harness 约束体系

Harness主要作用：
约束 Agent 行为，使其在正确边界内执行任务

实践中遇到的典型问题

验证报告显示通过，但修改代码从未被执行，Agent 编造技术解释而非调查

Agent 声称成功完成某项任务，但实际并未按要求完成

上下文约束

问题： 信息越多越容易忽略关键指令

方案： AGENT.md，Skill主体精简，拆分流程至subagent执行

角色约束

问题： Agent 跳过验证或越界改代码

方案： 按功能与权限拆角色，Hook 自动阻断越界

状态约束

问题： 上下文压缩丢失决策和调试记录

方案： 状态共享文档，归档+按需索引

流程约束

问题： Agent 跳步执行，不遵循规定的流程

方案： 强制门禁+自验证检查

MECHANISM & RULES

Hook 机制与反馈

Hook 机制（文字约束 → 代码级阻断）

- **角色越界保护：** analyzer/reviewer 修改模型代码时直接阻断
- **改代码前必读 progress：** 未读 progress.md 就改代码时阻断
- **自验证完整性检查：** implementer 结束时缺少自验证项则打回
- **外循环重试上限：** 同一阶段超 5 轮循环则阻断
- **长时间任务提醒：** 超 60 分钟周期提醒偏离检查

缺陷反馈

1. 每次优化总结报告记录 skill 流程缺失 / 约束缺失 / 参考过时等问题
2. 审阅后更新 skill 及编排流程
3. 当前规则和 Hook 均由此反馈循环驱动产生

失败驱动逐渐完善约束

1. 主 agent 只编排不改代码
2. 自验证缺失打回
3. 验证前确认代码被执行
4. 异常数据必须调查
5. 派发 prompt 简洁
6. 基线/warmup 标准
7. 架构一致性检查

使用方式

03

USAGE

使用方式

1 方式一：infer仓库直接使用

通过 cann-recipes-infer 仓库直接使用，Skill 已内置

1、克隆仓库： git clone https://gitcode.com/cann/cann-recipes-infer.git

2、切换工作目录： cd cann-recipes-infer

3、拉起Agent服务： opencode 或 claude

2 方式二：CANNBot 安装

通过 CANNBot 安装到已有项目

1、克隆仓库： git clone https://gitcode.com/cann/skills.git

2、切换工作目录： cd skills/teams/infer-model-optimize-team

3、执行初始化脚本： bash init.sh project opencode
或 bash init.sh project claude

4、拉起Agent服务： opencode 或 claude

启动模型优化 Agent Skill 方式

全流程优化（推荐）：“帮我完成 XX 模型NPU端到端适配及优化”

单阶段优化：“对 XX 模型单独做融合算子优化”

Demo演示

未来规划

04

ROADMAP

未来规划

Q1已完成

基础优化闭环

Q2

流程完善及规模扩展

更多模型适配验证；多流、SuperKernel 等更多高阶特性加入主流程

Q2

性能分析能力集成

完成优化特性使能 → Profiling 分析 → 定向调优的闭环

Q3

知识库自迭代

建立反馈知识库， 探索AVO自进化，实现 Skill 自动迭代更新

Q3

新方案设计能力拓展

完成融合算子方案设计和算子自动生成；支持现有与新增量化方案分析及实现

欢迎社区参与共建

cann-recipes-infer

NPU模型推理优化开源仓

CANNBot（cann/ skills）

Agent Skills 工具开源仓

CANNBot 与 Skill 开源交流群

试用　提 issue　补充案例与断点　参与社区讨论

Thanks!

访问CANN开源社区

关注昇腾CANN公众号